Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Lojik Devreler ve Tasarım

Bahar 2023

Dr. Öğr. Ü. İhsan Çiçek

5. Karnough Haritalarla Sadeleştirme, Birleşik Lojik Yapıtaşları ve Zamanlama

BOOLEAN LOGIC MINIMIZATION

We can use Boolean algebra postulates to reduce complexity of logic expressions while preserving the same functionality. For instance let us take a closer look at the following truth table:

A	B	C	T1	T2
0	0	0	1	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	0	1

There are three main methods for minimization

–Algebraic minimization

–Karnaugh Map minimization

–Quine-McCluskey (tabular) minimization

$T1 = \sum m(0,1,2)$, and $T2 = \sum m(3,4,5,6,7)$,

$T1 = A'B'C' + A'B'C + A'BC' = A'B'(\mathbf{C + C'}) + A'BC'$
$= A'(B' + BC') = A'(\mathbf{(B + B')\,(B'+C')}) = \mathbf{A'(B' + C')}$

$T2 = T1' = A'BC + AB'C' + AB'C + ABC' + ABC$
$= BC(\mathbf{A + A'}) + AB'(\mathbf{C' + C}) + AB(\mathbf{C' + C})$
$= BC + AB' + AB = BC + A(\mathbf{B' + B}) = \mathbf{A + BC}$

GATE LEVEL MINIMIZATION

- Karnaugh Maps (K-Maps) are an alternative for presenting Boolean functions which can be used to minimize them.
- Easy conversion from truth table/SoP to minimized SoP form.
- Simple rules are used to perform the minimization process.
- Faster than minimization using algebraic methods.
- Karnaugh maps are effective for functions with inputs < 6.
- Implicants, Prime Implicants, Essential Prime Implicants.
- Don't cares to simplify functions

KARNAUGH MAPS

- K-Map is an alternative representation of a Boolean function in which all rows of the truth table is represented with a square which corresponds to a minterm.
- Easy to convert between truth table, K-Map and Sum of Products.
- In an unoptimized form,

of 1s in K-map = # of minterms in truth table

Example: F = ∑m(0,1) = x'y + x'y'

x \ y	0	1
0	x'y'	x'y
1	xy'	xy

x \ y	0	1
0	1	1
1	0	0

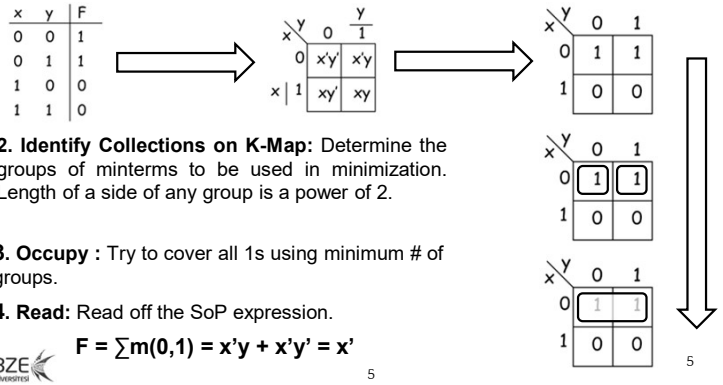
x	y	F
0	0	1
0	1	1
1	0	0
1	1	0

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

KARNAUGH MAPS

- K-Map is a graphical tool for assisting in the general simplification procedure.

1. Enter the function on K-Map: For each row/minterm that makes **F = 1**, place a **1** in the corresponding square.



2. Identify Collections on K-Map: Determine the groups of minterms to be used in minimization. Length of a side of any group is a power of 2.

3. Occupy : Try to cover all 1s using minimum # of groups.

4. Read: Read off the SoP expression.

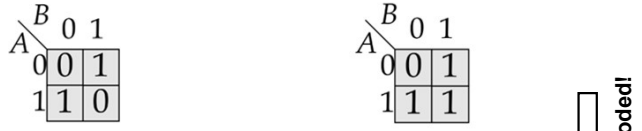
F = ∑m(0,1) = x'y + x'y' = x'

5

KARNAUGH MAPS

Two Variable Maps

2 bit K-Map is composed of one variable for a row and a column



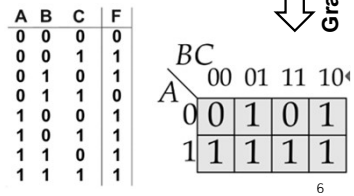
F_1 = ∑m(1,2)
= AB' + A'B

F_2 = ∑m(1,2,3)
= A'B + AB' + AB

Three Variable Maps

3 bit K-Map is composed of one variable for a row and two variables for a column.

F_3 = ∑m(1,2,4,5,6,7)
= A'B'C + A'BC' + AB'C' + AB'C
+ ABC' + ABC

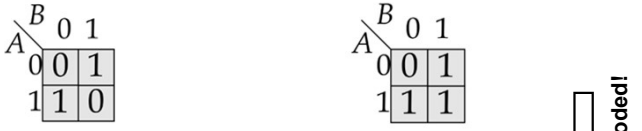


6

KARNAUGH MAPS

Two Variable Maps

2 bit K-Map is composed of one variable for a row and a column



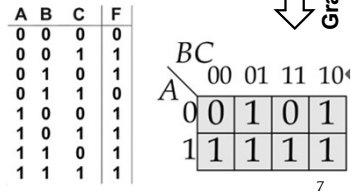
F_1 = ∑m(1,2)
= AB' + A'B

F_2 = ∑m(1,2,3)
= A'B + AB' + AB

Three Variable Maps

3 bit K-Map is composed of one variable for a row and two variables for a column.

F_3 = ∑m(1,2,4,5,6,7)
= A'B'C + A'BC' + AB'C' + AB'C
+ ABC' + ABC



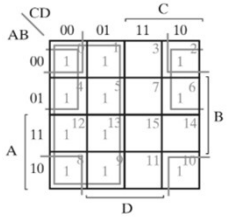
7

KARNAUGH MAPS

Four Variable Maps

4 bit K-Map is composed of two variable for a row and two variables for a column.

- Only one bit changes between adjacent, so Gray Encoding is used in K-Maps, this allows us to see the transitions of variables.



F = ∑m(0,1,2,4,5,6,8,9,10,12,13)

- Although 5, 6 variable maps are available, it is hard to use them in minimization, as map gets larger, it becomes harder to identify the adjacencies.

8

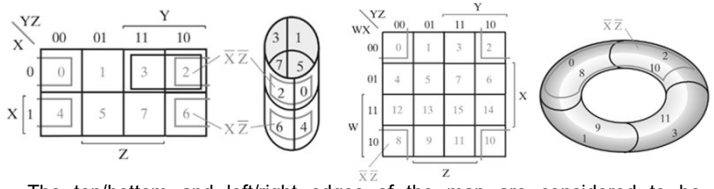
Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

RULES FOR KARNAUGH MAPS

- Boolean expressions can be minimized by grouping **1**s. **1** cells can only be grouped with adjacent **1** cells that are immediately above, below, left or right of that cell; no diagonal grouping.
- Each group represents a minterm reduction
- After covering all **1**s, minimized Sum of Products form is obtained.
- Every cell with **1** must be included at least once.
- The largest possible power of 2 rectangle/square must be covered. Rectangle/squares can overlap
- The **1**s must be covered using smallest possible number of rectangles.
- Every cell is a neighbor of the cell next to it.

RULES FOR KARNAUGH MAPS

- The map can be wrapped in cylindrical form along a vertical axis or a horizontal axis. Hence, squares at the edges are neighbors.



- The top/bottom and left/right edges of the map are considered to be continuous, so larger groups can be made by grouping cells across the top and bottom or left and right edges of the map.

Karnaugh Maps (K-Maps)

- We saw that boolean expressions can be minimized by combining terms and utilizing boolean algebra.
- Karnaugh maps (K-maps) are **a graphical method** for simplifying Boolean equations.
- It utilizes the simple rule AB + $A\bar{B}$ = A

A	B	C	Y
0	0	0	**1**
0	0	1	**1**
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Y / C \ AB	00	01	11	10
0	1	0	0	0
1	1	0	0	0

Y / C \ AB	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$

K-Map - Preparation

- Draw the graph and write inputs in **Gray encoding** order.
  - For 2 input circuit: 2x2 squares
  - For 3 input circuit: 2x4 squares
  - For 4 input circuit: 4x4 squares
- Fill out the graph using the truth table.
- Each square is the minterm of the corresponding inputs

A	B	C	Y
0	0	0	**1**
0	0	1	**1**
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Y / C \ AB	00	01	11	10
0	1	0	0	0
1	1	0	0	0

Y / C \ AB	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$

K-Map - Simplification

- Circle 1's in adjacent squares **according the K-Map rules.**
- In Boolean expression, include only literals whose **true and complement form** and **NOT** in the circle.

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Y \ AB	00	01	11	10
C = 0	1	0	0	0
C = 1	1	0	0	0

$$Y = \overline{A}\,\overline{B}\,\overline{C} + \overline{A}\,\overline{B}C = \overline{A}\,\overline{B}(\overline{C} + C) = \overline{A}\,\overline{B}$$

GEBZE TEKNİK ÜNİVERSİTESİ 13

13

K-Map - Rules

- Every 1 must be circled **at least** once.
- Each circle must span a power of 2 (i.e. 1, 2, 4) squares in **each direction.**
  - 1x1, 1x2, 2x1, 2x2, 4x1, 1x4, 4x2, 2x4, 4x4
- Each circle must be **as large as** possible.
- A circle may wrap around the edges of the K-Map.
- A **don't care (X)** is circleed **only if** it helps minimize the equation.

GEBZE TEKNİK ÜNİVERSİTESİ 14

14

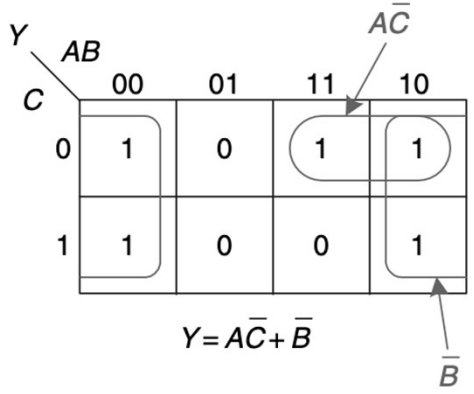
Example

Y \ AB	00	01	11	10
C = 0	1	0	1	1
C = 1	1	0	0	1

GEBZE TEKNİK ÜNİVERSİTESİ 15

15

Example



GEBZE TEKNİK ÜNİVERSİTESİ 16

16

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

K-Map Example 1

A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Y \ AB	00	01	11	10
C = 0				
C = 1				

17

K-Map Example 2

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Y \ AB	00	01	11	10
CD = 00				
CD = 01				
CD = 11				
CD = 10				

18

K-Map Example 2 - Solution

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Y \ AB	00	01	11	10
CD = 00	1	0	0	1
CD = 01	0	1	0	1
CD = 11	1	1	0	0
CD = 10	1	1	0	1

$Y = \bar{A}\bar{C} + \bar{A}BD + A\bar{B}\bar{C} + \bar{B}\bar{D}$

19

K-Map Example 3

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Y \ AB	00	01	11	10
CD = 00				
CD = 01				
CD = 11				
CD = 10				

20

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

K-Map Example 4

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

Y / CD \ AB	00	01	11	10
00				
01				
11				
10				

GEBZE TEKNİK ÜNİVERSİTESİ

21

21

K-Map Example 4 - Solution

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

Y / CD \ AB	00	01	11	10
00	1	0	X	1
01	0	X	X	1
11	1	1	X	X
10	1	1	X	X

$Y = A + \bar{B}\bar{D} + C$

GEBZE TEKNİK ÜNİVERSİTESİ

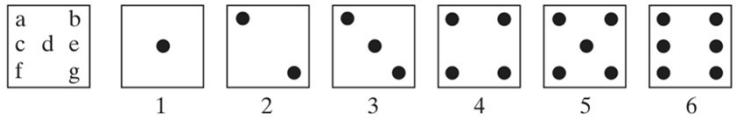
22

22

EXAMPLE: DR. CICEK'S Electonic DICE

Dr. Cicek's electronic-dice display a random number from the set **{1,2,3,4,5,6}** when it is rolled. The display circuit will have three inputs **x,y,z** for the number to be displayed and seven output drivers for LEDs **a,b,c,d,e,f,g**. The dice display patterns:





GEBZE TEKNİK ÜNİVERSİTESİ

23

23

EXAMPLE: DR. CICEK'S Electonic DICE

a. Create a truth table that defines the desired logic functionality. Outputs that correspond to unused inputs will be all logic zeros.

b. Find the Boolean functions for each output function in the standard sum-of-products form.

c. Minimize output functions using K-map method.

d. Draw the gate level logic diagram of the display driver.

GEBZE TEKNİK ÜNİVERSİTESİ

24

24

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

EXAMPLE: DR. CICEK'S Electonic DICE

Let us define the truth table using the given data

x	y	z	F_a	F_b	F_c	F_d	F_e	F_f	F_g
0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	0	0
0	1	0	1	0	0	0	0	0	1
0	1	1	1	0	0	1	0	0	1
1	0	0	1	1	0	0	0	1	1
1	0	1	1	1	0	1	0	1	1
1	1	0	1	1	1	0	1	1	1
1	1	1	0	0	0	0	0	0	0

$F_a = F_g = \sum m(2,3,4,5,6)$ $F_b = F_f = \sum m(4,5,6)$ $F_c = F_e = \sum m(6)$ $F_d = \sum m(1,3,5)$

25

EXAMPLE: DR. CICEK'S Electonic DICE

(b) Derive the Boolean functions for each output function in the standard sum-of-products form.

$F_a = F_g = \sum m(2,3,4,5,6) = x'yz' + x'yz + xy'z' + xy'z + xyz'$

$F_b = F_f = \sum m(4,5,6) = xy'z' + xy'z + xyz'$

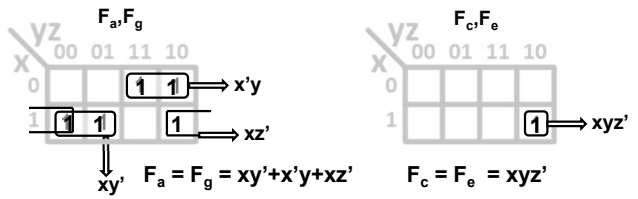
$F_c = F_e = \sum m(6) = xyz'$

$F_d = \sum m(1,3,5) = x'y'z + x'yz + xy'z$

26

EXAMPLE: DR. CICEK'S Electonic DICE

(c) Minimize all output functions using Karnaugh map method.

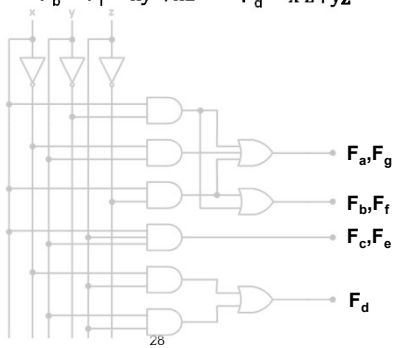


27

EXAMPLE: DR. CICEK'S Electonic DICE

(d) Draw the gate level logic diagram of the display driver, assume that the complement of the inputs are not readily available. You can use inverter gates to generate the complements.

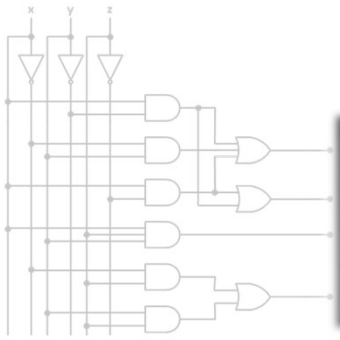
$F_a = F_g = xy'+x'y+xz'$ $F_b = F_f = xy'+xz'$ $F_d = x'z+yz'$
$F_c = F_e = xyz'$



28

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

EXAMPLE: DR. CICEK'S Electonic DICE



Dr. Cicek's Electronic Dice in Action…



DON'T CARE CONDITIONS

Incompletely specified functions the function is not specified for certain variable value combinations:

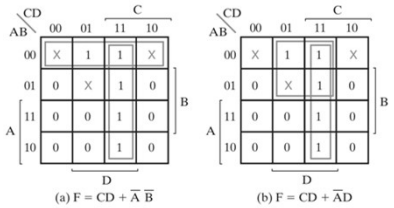
- The input combinations never occur.
- The input combinations are expected to occur, but we do not care what the outputs are in response to these combinations.

The unspecified minterms = "don't-care conditions"
X is used to mean that we do not care whether the value of 0 or 1 is assigned for that minterm.

DON'T CARE CONDITIONS

Each X on the map can be a 1 or 0 depending on whichever is more convenient in minimization.

$$F(A, B, C, D) = \Sigma m(1, 3, 7, 11, 15)$$
$$d(A, B, C, D) = \Sigma m(0, 2, 5)$$



$$F = D(\overline{A} + C)$$

EXAMPLE: DR. CICEK'S E-DICE REVISITED

Let us define the truth table using the given data

x	y	z	F_a	F_b	F_c	F_d	F_e	F_f	F_g
0	0	0	X	X	X	X	X	X	X
0	0	1	0	0	0	1	0	0	0
0	1	0	1	0	0	0	0	0	1
0	1	1	1	0	0	1	0	0	1
1	0	0	1	1	0	0	0	1	1
1	0	1	1	1	0	1	0	1	1
1	1	0	1	1	1	0	1	1	1
1	1	1	X	X	X	X	X	X	X

$F_a = F_g = \Sigma m(2,3,4,5,6)$
$F_b = F_f = \Sigma m(4,5,6)$
$F_c = F_e = \Sigma m(6)$
$F_d = \Sigma m(1,3,5)$

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

EXAMPLE: DR. CICEK'S E-DICE REVISITED

b. Derive the Boolean functions for each output function in the standard sum-of-products form.

$F_a = F_g = \sum m(2,3,4,5,6) = x'yz' + x'yz + xy'z' + xy'z + xyz'$

$F_b = F_f = \sum m(4,5,6) = xy'z' + xy'z + xyz'$

$F_c = F_e = \sum m(6) = xyz'$

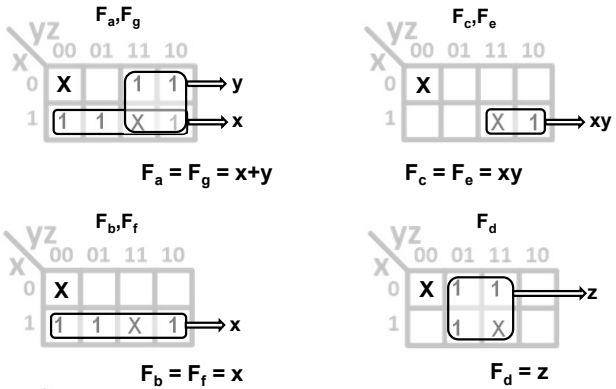
$F_d = \sum m(1,3,5) = x'y'z + x'yz + xy'z$

For all functions Don't Cares are

$F_x = \sum m(0,7) = x'y'z' + xyz$

EXAMPLE: DR. CICEK'S E-DICE REVISITED

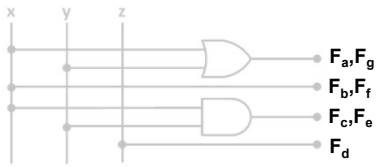
c. Minimize all output functions using Karnaugh map method.



EXAMPLE: DR. CICEK'S E-DICE REVISITED

(d) Draw the gate level logic diagram of the display driver, assume that the complement of the inputs are not readily available. You can use inverter gates to generate the complements.

$F_a = F_g = x + y$ $F_b = F_f = x$ $F_d = z$
$F_c = F_e = xy$



Only two gates! Compare it to the previous solution which required 12 gates!

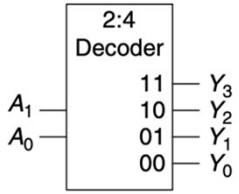
Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Combinational Building Blocks

- Combinational Logic is often grouped into larger building blocks to build more complex systems.
- These include
  - decoders
  - encoders
  - multiplexers
  - demultiplexers

Decoders

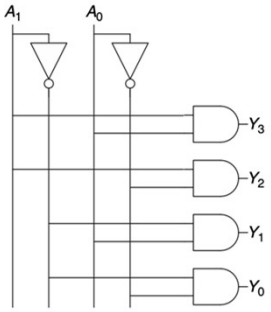
- Has N inputs, and 2^N outputs. **N:2^N Decoder**
- Mostly used for driving large number of components with minimum number of pins.
- **One-hot** outputs - one-hot means exactly one output is HIGH (hot) at a given time. (usually)

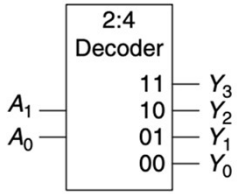


A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

2:4 Decoder

- Using logic gates we can implement a 2:4 decoder as



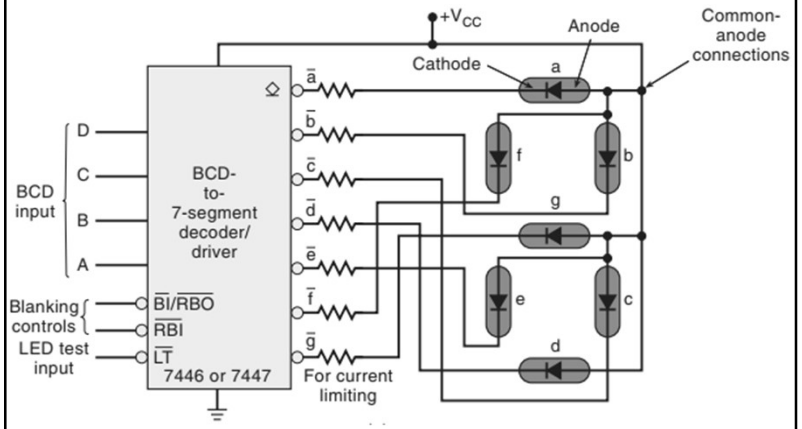


A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

Decoder Applications

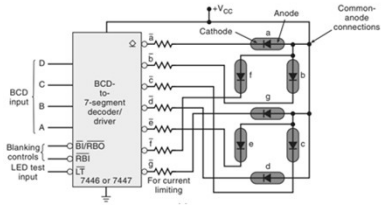
- Decoders are used whenever an output or a group of outputs are to be activated only on the occurrence of a specific combination of input levels.
- Usually come after counters
- BCD to Seven Segment decoder
- BCD to Decimal decoder
- Memory systems (for addressing)

BCD to Seven Segment Decoder



41

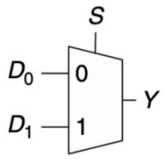
BCD to SSD Decoder Example - 2



42

Multiplexers (MUX)

- Chooses an output from several possible inputs based on the value of a **select** line.
- One of the most commonly used combinational circuits.
- Can also called **a data selector**.
- **N:1 Mux** or **Multiplexer**



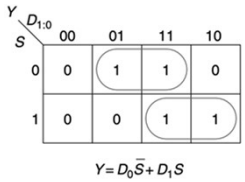
S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

43

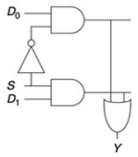
Multiplexer Implementations



Using Logic Gates

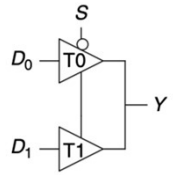


$Y = D_0\bar{S} + D_1S$



Using tristate buffers

- For an N-input Mux, use N tri-states
- Turn on exactly one to select the appropriate input



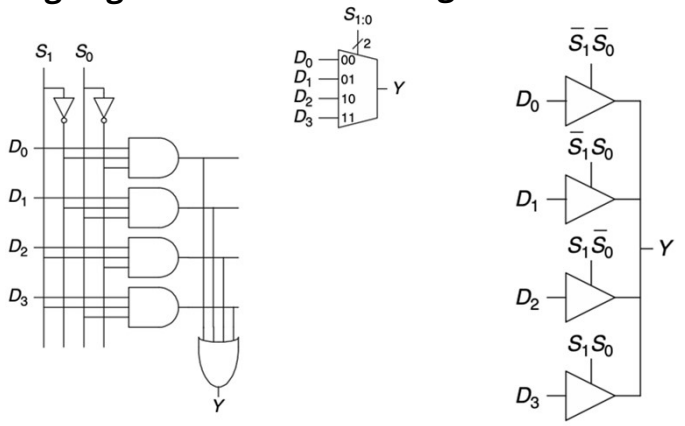
$Y = D_0\bar{S} + D_1S$

44

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

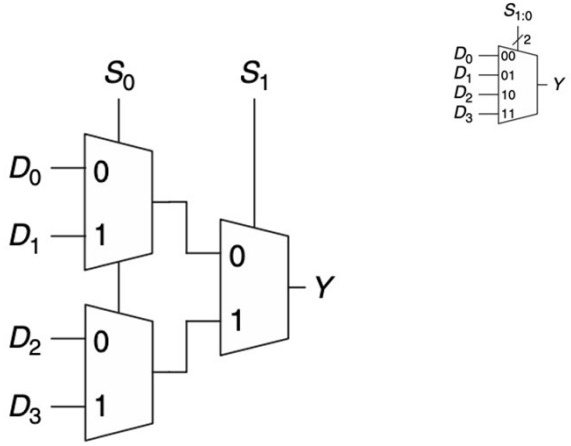
Wider Multiplexers - 4:1 Mux

Using Logic Gates **Using tristate buffers**



45

4:1 Mux from 2:1 Muxes (Hierarchical)



46

Multiplexer Applications

Used in various designs. Examples include

- **Data Routing** - multiple sources with single data destination
- **Parallel-to-Serial Conversion** - parallel data is converted to serial data to be transmitted over single line
- **Operation Sequencing** - completion of multiple steps one at a time

Note: There is also a Demultiplexer (Demux) which works and looks exactly the opposite of a Mux

47

PRIMITIVE LOGIC FUNCTIONS

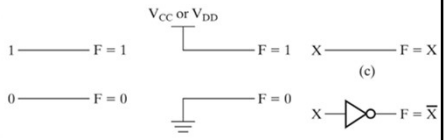
VALUE FIXING, TRANSFERRING AND INVERTING

For a single-bit output function with a single variable X, 4 different functions are possible.

- **Value Fixing**: Assign a constant value 0 or 1 to the function output.
- **Value Transferring:** Assign the input X to the function output.
- **Value Inverting:** Assign the inverted input X' to the function output.

Functions of One Variable

X	F = 0	F = X	F = $\overline{X}$	F = 1
0	0	0	1	1
1	0	1	0	1



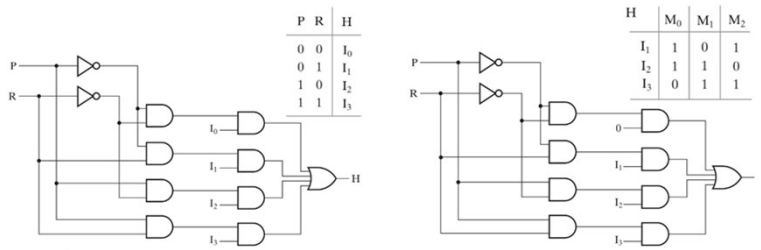
48

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

PRIMITIVE LOGIC FUNCTIONS

VALUE FIXING, TRANSFERRING AND INVERTING

- **Value Fixing** involves replacing variables with constant values 1 and 0. It may be permanent(can never be changed) or temporary(can be changed by programming).
- In programmable logic devices, any logic function that is within the capacity of the programmable device can be implemented by fixing a set of values.



EXCLUSIVE OR OPERATOR AND GATE

XOR gates implement the XOR operator. $X \oplus Y = X\overline{Y} + \overline{X}Y$

XNOR gates implement the XNOR operator. $\overline{X \oplus Y} = XY + \overline{X}\,\overline{Y}$

Following Identities applies to XOR

$X \oplus 0 = X \qquad X \oplus 1 = \overline{X}$

$X \oplus X = 0 \qquad X \oplus \overline{X} = 1$

$X \oplus \overline{Y} = \overline{X \oplus Y} \qquad \overline{X} \oplus Y = \overline{X \oplus Y}$

XOR is commutative and associative

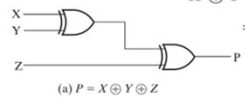
$A \oplus B = B \oplus A$

$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$

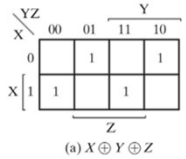
Odd Function: 3 variable XOR function = 1 only when one or three variables are 1. So called the odd function. **Remember the parity codes?**

$X \oplus Y \oplus Z = (X\overline{Y} + \overline{X}Y)\overline{Z} + (XY + \overline{X}\,\overline{Y})Z$

$= X\overline{Y}\,\overline{Z} + \overline{X}Y\overline{Z} + \overline{X}\,\overline{Y}Z + XYZ$



(a) $P = X \oplus Y \oplus Z$



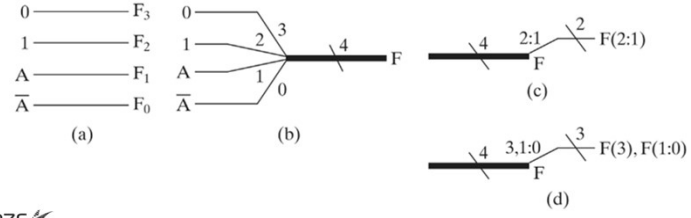
(a) $X \oplus Y \oplus Z$

PRIMITIVE LOGIC FUNCTIONS

MULTIPLE-BIT FUNCTIONS

The logic functions can be applied to multiple bits on a bitwise basis. We can think of these multiple-bit functions as vectors of single-bit functions.

EXAMPLE: Assume we have 4 functions F3,F2,F1,F0. If MSB = F3, and LSB = F0 then F=(F3,F2,F1,F0) is the multi-bit vector function. This function is referred as F(3:0). In diagrams, thick lines represent multi-bit vectors and the bit-width is indicated by a slash cross the line and a number.



PRIMITIVE LOGIC FUNCTIONS

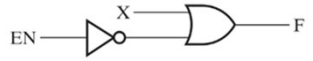
ENABLE FUNCTION

Enabling permits an input signal to pass through to an output. It can replace the input signal with a fixed output value, either 0 or 1.

EXAMPLE: if EN is 1, the input X reaches the output (**enabled**), but if EN is 0, the output is fixed at 0 (**disabled**).

If the disabled value is 1, then the input signal X is ORed with the complement of the EN signal. It is also possible for each of the circuits to be modified to invert the EN input, so that EN = 0 enables X to reach the output and EN = 1 blocks X.





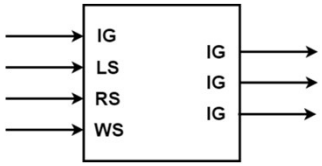
Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

PRIMITIVE LOGIC FUNCTIONS

ENABLE FUNCTION, AN EXAMPLE

In cars, the lights, radio, and power windows operate only if the ignition switch is turned on. Let us model this automotive subsystem:

INPUTS: **Ignition switch IG**: 0 if OFF and 1 if ON
Light switch LS: 0 if OFF and 1 if ON
Radio switch RS: 0 if OFF and 1 if ON
Power window switch WS: 0 if OFF and 1 if ON



OUTPUTS: **Lights L**: 0 if OFF and 1 if ON

Radio R: 0 if OFF and 1 if ON

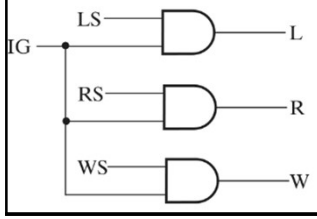
Power windows W: 0 if OFF and 1 if ON

GEBZE TEKNİK ÜNİVERSİTESİ

PRIMITIVE LOGIC FUNCTIONS

Truth Table For Enabling Application

Input Switches				Accessory Control		
IS	LS	RS	WS	L	R	W
0	X	X	X	0	0	0
1	0	0	0	0	0	0
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	0	1	1
1	1	0	0	1	0	0
1	1	0	1	1	0	1
1	1	1	0	1	1	0
1	1	1	1	1	1	1



ENABLE FUNCTION, AN EXAMPLE

If the ignition switch **IS** is OFF (0), all accessories are OFF (0) regardless of their set values.

In the first input row of the table with the use of (**X**s), the truth table is compacted using 9 rows instead of 16-rows.

(**X**s) in outputs typically represent don't-care conditions,

(Xs**)** in input columns are used to represent product terms that are not minterms. If the corresponding bit in the input combination is an **(X)**, then the variable does not appear in the product term.

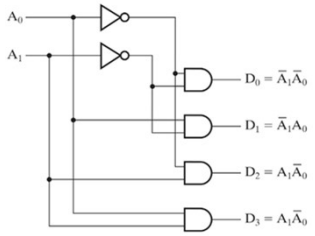
PRIMITIVE LOGIC FUNCTIONS

DECODING FUNCTION: n-bit binary code can represent $m \le 2^n$ distinct elements of coded information (minterms). Decoding is the conversion of an n-bit valid input code to an m-bit output unique code with $n \le m \le 2^n$

The combinational logic blocks that implement decoding are called n–to–m-line decoders, where $m \le 2^n$. Their purpose is to generate $m \le 2^n$ minterms from the n input variables.

EXAMPLE: For n=2, m=4 we get 2-to-4-line decoder.

A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



GEBZE TEKNİK ÜNİVERSİTESİ

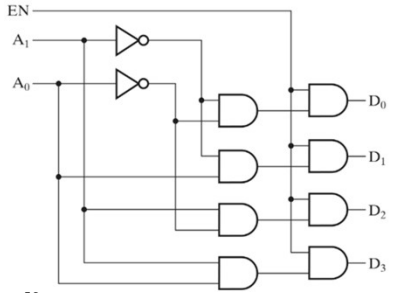
PRIMITIVE LOGIC FUNCTIONS

DECODING WITH ENABLE FUNCTION

n–to–m-line decoder with enable can be implemented by attaching m enabling circuits to the decoder outputs. In large decoders (n > 4), the gate-input cost can be reduced by placing the enable circuits on the inputs to the decoder.

EXAMPLE: 2-to-4-line decoder with enable.

EN	A_1	A_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1



GEBZE TEKNİK ÜNİVERSİTESİ

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

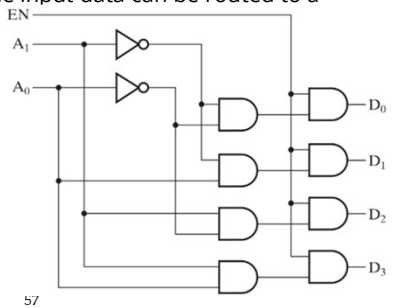
PRIMITIVE LOGIC FUNCTIONS

DECODING WITH ENABLE FUNCTION = DEMULTIPLEXER

(selection)' = distribution, (info received from a single input is transmitted to one of the 2^n possible outputs controlled by n selection inputs. The circuit which implements the distribution is called a demultiplexer. Thus, a 2–to–4-line decoder with enable is a 1–to–4-line demultiplexer! For the demultiplexer, input EN is the data input, while the other inputs are used for output channel selection. So the input data can be routed to a selected output

EXAMPLE: 2-to-4-line **DEMUX**

EN	A_1	A_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1



GEBZE TEKNİK ÜNİVERSİTESİ

57

57

PRIMITIVE LOGIC FUNCTIONS

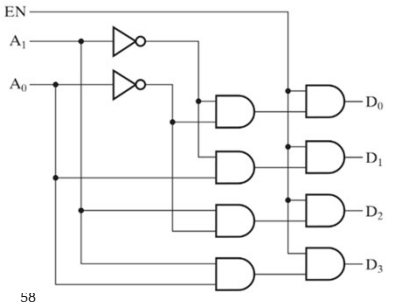
DECODING WITH ENABLE FUNCTION = DEMULTIPLEXER

Although the decoders and demuxes have different applications, their logic diagrams are exactly the same!

For this reason, a decoder with enable input is referred to as a decoder/ demultiplexer

EXAMPLE: 2-to-4-line **DEMUX = 2-to-4 line DECODER with EN = Data**

EN	A_1	A_0	D_0	D_1	D_2	D_3
0	X	X	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1



GEBZE TEKNİK ÜNİVERSİTESİ

58

58

PRIMITIVE LOGIC FUNCTIONS

DECODER BASED COMBINATIONAL CIRCUITS

A decoder provides the 2^n minterms for n input variables. You can use a decoder to generate the minterms and combine them with an external OR gate to form any SoP implementation. Any combinational circuit with n inputs and m outputs can be implemented with an n–to–2^n-line decoder + m OR gates.

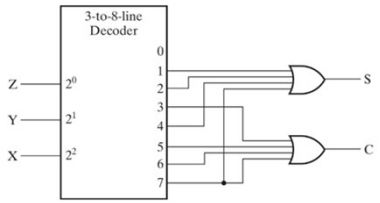
EXAMPLE: Decoder and Or-gate implementation of a Binary adder

Truth Table for 1-Bit Binary Adder

X	Y	Z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Z = Cin, S = sum, C = Cout From Table

C = Σm(3,5,6,7), S = Σm(1,2,4,7),



GEBZE TEKNİK ÜNİVERSİTESİ

59

59

PRIMITIVE LOGIC FUNCTIONS

DECODER BASED COMBINATIONAL CIRCUITS

A function with a long list of minterms requires an OR gate with a large number of inputs.

A function having a list of k minterms can be expressed in its complement form with 2^n - k minterms. If the number of minterms in a function F is greater than that of F', then the complement F', can be expressed with fewer minterms. In such a case, it is advantageous to use a NOR gate instead of an OR gate.

The OR portion of the NOR gate produces the logical sum of the minterms of F'. The output bubble of the NOR gate complements this sum and provides F.

The decoder methodcan be used to implement any combinational logic function, but it may provide the best solution, particularly if the combinational circuit has many outputs based on the same inputs and each output function is expressed with a small number of minterms.

GEBZE TEKNİK ÜNİVERSİTESİ

60

60

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

PRIMITIVE LOGIC FUNCTIONS

ENCODING FUNCTION

An encoder(Digital function that performs the inverse operation of a decoder) has $m \leq 2^n$ input lines and n output lines. The output lines generate the binary code corresponding to the input value.

The encoder just defined has the limitation that **only one input can be active at any given time.** if two or more inputs are active simultaneously, the output produces an incorrect combination.

EXAMPLE: For $2^n=8$, m=8 we get 8-to-3-line encoder using 4 input OR gates.

A0 = D1 + D3 + D5 + D7
A1 = D2 + D3 + D6 + D7
A2 = D4 + D5 + D6 + D7

Truth Table for Octal-to-Binary Encoder

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

PRIMITIVE LOGIC FUNCTIONS

ENCODING FUNCTION

For example, if D3 and D6 are 1 simultaneously, the output of the encoder will be 111, because all the three outputs are equal to 1. This represents neither a binary 3 nor a binary 6.

Truth Table for Octal-to-Binary Encoder

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

To resolve this ambiguity, some encoder circuits must establish an input priority to ensure that only one input is encoded. If we establish a higher priority for inputs with higher subscript numbers, and if both D3 and D6 are 1 at the same time, the output will be 110, because D6 has higher priority than D3.

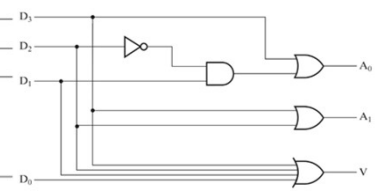
PRIMITIVE LOGIC FUNCTIONS

PRIORITY ENCODER FUNCTION

A priority encoder is a combinational circuit that implements a priority function. If two or more inputs are equal to 1 at the same time, the input having the highest priority takes precedence. **(X**s**)** in the outputs are don't-care conditions, **(X**s**)** in inputs are used to represent product terms that are not minterms.

Truth Table of Priority Encoder

Inputs				Outputs		
D_3	D_2	D_1	D_0	A_1	A_0	V
0	0	0	0	X	X	0
0	0	0	1	0	0	1
0	0	1	X	0	1	1
0	1	X	X	1	0	1
1	X	X	X	1	1	1



A0 = D3 + D1 D2'
A1 = D2 + D3
V = D0 + D1 + D2 + D3

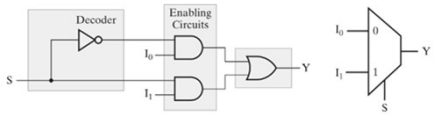
PRIMITIVE LOGIC FUNCTIONS

SELECTION FUNCTION : MULTIPLEXERS - MUX

A multiplexer is a combinational circuit that selects binary information from one of many input lines and directs the information to a single output line. There are 2^n input lines and n selection inputs whose bit combinations determine which input is selected. A multiplexer is also called a data selector, since it selects one of many inputs and routes the selected input data to the output.

Truth Table for 2–to–1-Line Multiplexer

S	I_0	I_1	Y
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1



$Y = s'I_0 + sI_1$

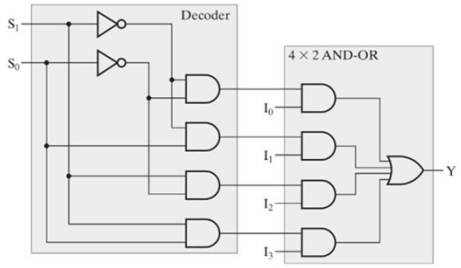
Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

PRIMITIVE LOGIC FUNCTIONS

SELECTION FUNCTION : MULTIPLEXERS - MUX

EXAMPLE: 4-to-1 MUX: Y depends on I_0, I_1, I_2, I_3 and s_1, s_0. By placing the values of I_0 - I_3 in the Y column, a condensed truth table can be formed.

S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



$Y = (s_1's_0')I_0 + (s_1's_0)I_1 + (s_1s_0')I_2 + (s_1s_0)I_3$

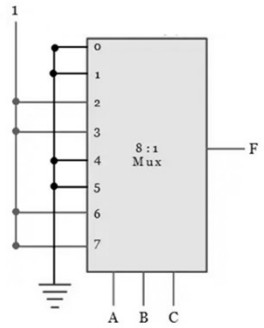
65

PRIMITIVE LOGIC FUNCTIONS

A multiplexer can be used to implement Boolean logic functions.

EXAMPLE: Implement the boolean expression F(A, B, C) = ∑ m(2, 3, 6, 7) using a multiplexer.

3 inputs => 2^3 to 1 MUX can be used. inputs, corresponding to the minterms (2, 3, 6, 7) are connected to logic 1 and the remaining terms to logic 0.

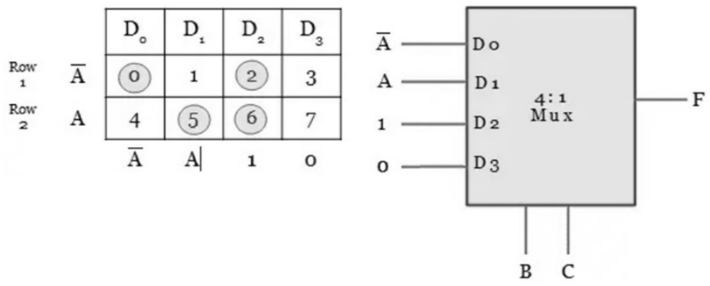


66

PRIMITIVE LOGIC FUNCTIONS

EXAMPLE: Implement the boolean expression F(A, B, C) = ∑ m(0, 2, 5, 6) using 4 : 1 multiplexer.

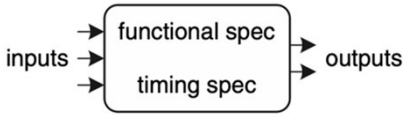
3 inputs but we have to use 2^2 to 1 MUX. From the given 3 variables, the 2 variables (B, C) can be used as selection inputs



67

Timing

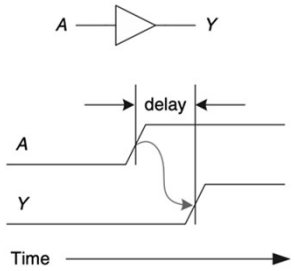
- Up until now, we have mostly been concerned about the **functionality** of the circuit, ideally using **fewest possible gates**.
- One of the most challenging issues in circuit design is **timing** - ***making a circuit run as fast as possible without breaking the functionality***.



68

Delay

- **Delay** - is ***the time to see the change in the input at the output.***
- Below is an example of the response of the output to a change in the input.

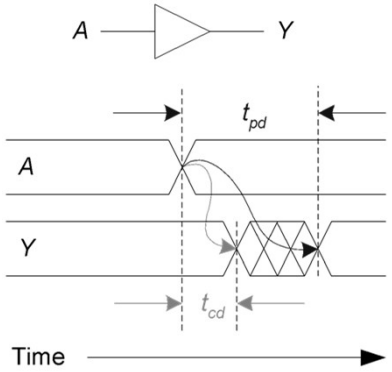


Rising and Falling Edge

- The change on the digital circuits does not happened instantaneously, but happens gradually.
  - The transition from LOW to HIGH is called **the rising edge**
  - The transition from HIGH to LOW is called **the falling edge**.

Propagation & Contamination Delay

- **Propagation delay :** t_{pd} = max delay from input to output
- **Contamination delay :** t_{cd} = min delay from input to output



Propagation & Contamination Delay

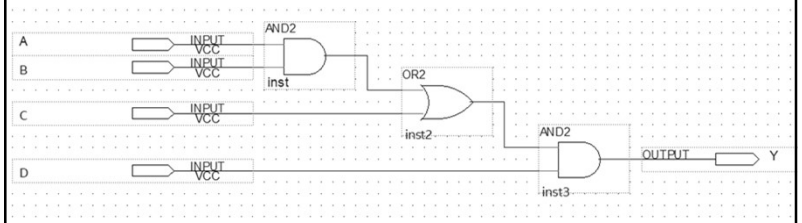
- Delay is caused by
  - Capacitance and resistance in a circuit
  - Speed of light limitation
- Reasons why t_{pd} and t_{cd} may be different:
  - Different rising and falling delays
  - Multiple inputs and outputs, some of which are faster than others
  - Circuits slow down when hot and speed up when cold

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Example

Consider the following circuit

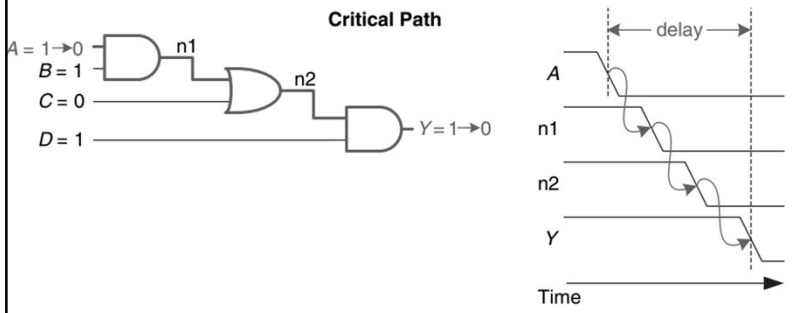
- What is the **critical (long) path**?
- What is the **short path**?



73

Critical (long) Path

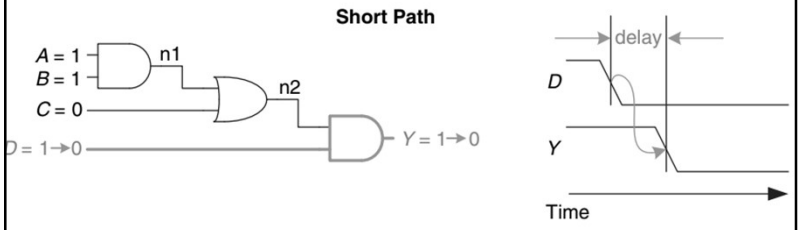
The critical path is the longest, therefore the slowest path because the input travels through three gates to the output.



74

Short Path

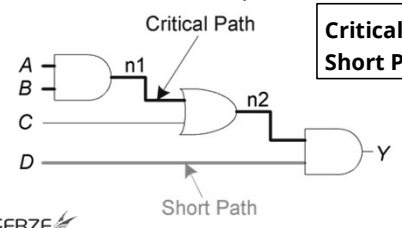
The short path is the shortest, therefore the fastests path because the input travels through one gate to the output.



75

Critical & Short Paths

- Propagation delay is the sum of the propagation delays through each element on the critical path.
- Contamination delay is the sum of the contamination delays through each element on the short path.

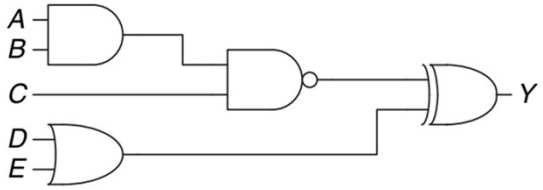


Critical (Long) Path: $t_{pd} = 2t_{pd_AND} + t_{pd_OR}$
Short Path: $t_{cd} = t_{cd_AND}$

76

Example

- Assume that each gate has propagation delay of 100 ps and contamination delay of 60 ps.
- Find the contamination and propagation delays.

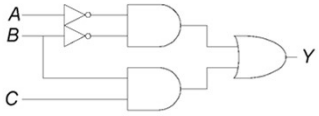


Glitches

- So far we've discussed the case when a single input transition causes a single output transition.
- It is possible that single input transition might cause multiple transitions which is called a **glitch** or **hazard**.
- They are usually not a problem (due to synchronous design conventions), but it is important to understand and identify them while looking at timing diagrams.

Glitch Example

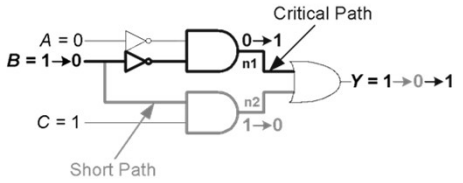
- Assume A = 0, B = 1, and C = 1. What happens B falls. (goes to 0)?

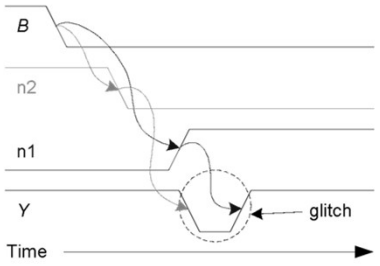


Y, AB / C	00	01	11	10
0	1	0	0	0
1	1	1	1	0

$Y = \bar{A}\bar{B} + BC$

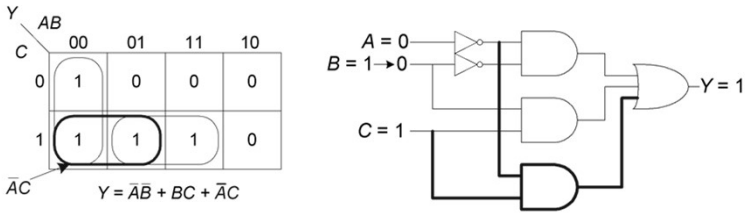
Glitch Example





Fixing the Glitch

We can fix this glitch by connecting two separate bubbles with additional bubble in the neighbors.



81

END OF LECTURE

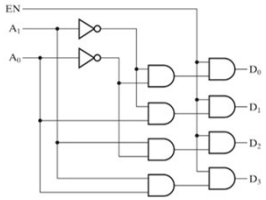
- Thanks

82

83

RUDIMENTARY LOGIC FUNCTIONS

2-to-4 DECODER WITH ENABLE VERILOG MODEL



VERILOG CODE (DATAFLOW)

```
module DEC2to4 (EN, A0, A1, D0, D1, D2, D3);
input EN, A0, A1;
output D0, D1, D2, D3;
assign D0 = EN & ~A1 & ~A0;
assign D1 = EN & ~A1 & A0;
assign D2 = EN & A1 & ~A0;
assign D3 = EN & A1 & A0;
endmodule
```

VERILOG CODE (STRUCTURAL)

```
module DEC2to4 (EN, A0, A1, D0, D1, D2, D3);
input EN, A0, A1;
output D0, D1, D2, D3;

wire A0_n, A1_n, N0, N1, N2, N3;

not g0(A0_n, A0), g1(A1_n, A1);

and g3(N0, A0_n, A1_n), g4(N1, A0, A1_n),
g5(N2, A0_n, A1), g6(N3, A0, A1), g7(D0, N0,
EN), g8(D1, N1, EN), g9(D2, N2, EN), g10(D3,
N3, EN);

endmodule
```


84